



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

ОТЧЕТ
ПО ПРЕДДИПЛОМНОЙ ПРАКТИКЕ
НА ТЕМУ:
«Плагин для VS-Code для автоматизации
формирования Unit тестов кроссплатформенных
приложений на Flutter»

Студент _____
(Группа)

(Подпись, дата)

(И.О. Фамилия)

Руководитель

(Подпись, дата)

(И.О. Фамилия)

2025 г.

АННОТАЦИЯ

Работа посвящена разработке плагина для интегрированной среды разработки Visual Studio Code, предназначенного для автоматической генерации модульных тестов для кроссплатформенных приложений на основе фреймворка Flutter. Рассматриваются подходы к генерации тестов для компонентов управления состоянием Cubit/BLoC с использованием моделей конечных автоматов и теории тестовых множеств, а также для DTO(data transfer object) и обычных методов/функций на языке Dart. Описываются архитектура плагина, процесс парсинга исходного кода, алгоритмы генерации тестовых сценариев и интеграция с инструментами тестирования Flutter, такими как пакет `bloc_test`.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 Обзор предметной области и постановка задачи	7
1.1 Используемые технологии	7
1.1.1 Visual Studio Code Extension API	7
1.1.2 TypeScript	7
1.1.3 Dart и Flutter Framework	8
1.1.4 Библиотека flutter_test	8
1.1.5 Библиотека bloc_test	9
1.2 Тестирование в разработке на Flutter	9
1.3 Проблемы ручного тестирования и необходимость автома- тизации	10
1.4 Постановка задачи разработки плагина	10
2 Проектирование плагина	12
2.1 Архитектура плагина	12
2.2 Основные интерфейсы и структуры данных	13
2.2.1 StateInfo — Информация о состояниях	13
2.2.2 EnhancedStateInfo — Расширенная информация о состояниях	13
2.2.3 DependencyInfo — Информация о зависимостях	14
2.2.4 EventInfo — Информация о событиях	14
2.2.5 ParsedMethod — Разобранный метод	14
2.2.6 EnhancedMethodInfo — Расширенная информация о методах	15
2.2.7 StateTransition — Переход между состояниями	15
2.2.8 GuardCondition — Guard условие	16
2.2.9 DartMethodContext — Контекст метода Dart	16
2.2.10 EmitInfo — Информация об emit вызове	16
2.2.11 FSM — Автомат конечных состояний	17
2.2.12 TestPath — Тестовый путь	17
2.2.13 ExecutionBranch — Ветка выполнения	17
2.2.14 Token и TokenType — Лексический анализ	18
2.2.15 DataItem — Информация о полях DTO	18

2.2.16	ASTNode — Узлы абстрактного синтаксического дерева	19
2.3	Основные функциональные компоненты	19
2.3.1	Анализатор кода	19
2.3.2	Построитель автоматов	20
2.3.3	Генератор тестовых путей	20
2.3.4	Генератор тестового кода	20
2.3.5	Интеграция с VS Code	21
2.3.6	Лексический анализатор Dart	21
2.4	Порядок выполнения	22
2.5	Взаимодействие с пользователем	22
2.6	Парсинг исходного кода	23
3	Реализация ключевых функций	25
3.1	Генерация тестов для Cubit/BLoC	25
3.1.1	Полный алгоритм работы с Cubit/BLoC	25
3.2	Генерация тестов для Data Transfer Object	27
3.2.1	Парсинг структуры Data Transfer Object	27
3.2.2	Генерация тестовых сценариев	27
3.2.3	Структура генерируемых тестов	28
3.3	Генерация тестов для методов/ивентов	28
3.4	Решение проблем и доработки	28
3.4.1	Создание стабильной архитектуры	29
3.4.2	Лексического анализа	29
3.4.3	Технические решения	29
3.4.4	Визуализация и пользовательский интерфейс	30
3.4.5	Качество кода	30
4	Инструкция пользователя	31
4.1	Установка плагина	31
4.2	Подготовка проекта	31
4.3	Использование плагина	32
4.4	Результат генерации	32
4.5	Возможности генерируемых тестов	32

5	Результаты работы	34
5.1	Результаты генерации тестов для Cubit	34
5.2	Результаты генерации тестов для BLoC	34
5.3	Результаты генерации тестов для Data Transfer Object	35
5.4	Качественные характеристики генерируемых тестов	36
5.5	Практическая ценность результатов	36
	ЗАКЛЮЧЕНИЕ	38
	ПРИЛОЖЕНИЕ А	40
	ПРИЛОЖЕНИЕ Б	43

ВВЕДЕНИЕ

Развитие мобильных и веб-приложений с использованием кроссплатформенных фреймворков, таких как Flutter, ставит перед разработчиками задачи не только быстрого создания функциональности, но и обеспечения высокого качества программного продукта. Модульное тестирование является ключевым элементом этого процесса, позволяя выявлять ошибки на ранних этапах и обеспечивать стабильность кодовой базы. Однако ручное написание тестов, особенно для компонентов со сложной логикой управления состоянием (например, Cubit в экосистеме BLoC), является трудоемким и подверженным ошибкам процессом.

Актуальность данной работы обусловлена необходимостью снижения трудозатрат на написание модульных тестов и повышения их полноты. Автоматизация генерации тестов позволяет разработчикам сосредоточиться на более сложных аспектах проектирования и реализации, при этом получая базовое тестовое покрытие для своих компонентов.

Целью настоящей работы является разработка плагина для среды VS Code, который автоматизирует процесс создания модульных тестов для Flutter-приложений, в частности для классов Cubit/BLoC, классов DTO и методов.

Основные задачи включают:

- Анализ структуры Dart-кода для извлечения информации о классах Cubit/BLoC, их состояниях, ивентах и методах, изменяющих состояние.
- Разработку алгоритма построения модели поведения компонента Cubit, приближенной к модели конечного автомата.
- Реализацию механизма генерации тестовых сценариев на основе анализа возможных переходов состояний Cubit/BLoC.
- Обеспечение генерации шаблонов тестов для отдельных методов/ивентов классов.
- Интеграцию плагина в среду разработки VS Code для удобного использования разработчиками.

1 Обзор предметной области и постановка задачи

1.1 Используемые технологии

Разработка плагина базируется на современном технологическом стеке, обеспечивающем эффективную интеграцию с экосистемой Flutter-разработки.

1.1.1 Visual Studio Code Extension API

Visual Studio Code был выбран в качестве целевой платформы для разработки плагина по следующим причинам:

- Широкое распространение среди Flutter-разработчиков: VS Code является одной из наиболее популярных IDE для разработки на Dart/Flutter благодаря официальной поддержке от команды Flutter.
- Развитая экосистема расширений: VS Code предоставляет мощный API для создания расширений, включающий возможности работы с файловой системой, редактором кода, пользовательским интерфейсом и интеграцией с внешними инструментами.
- Кроссплатформенность: расширения VS Code работают на Windows, macOS и Linux, что обеспечивает широкую доступность плагина.
- Интеграция с инструментами разработки: встроенная поддержка терминала, системы контроля версий и отладчика упрощает процесс разработки и тестирования.

1.1.2 TypeScript

TypeScript выбран в качестве основного языка разработки плагина:

- Статическая типизация: TypeScript обеспечивает проверку типов на этапе компиляции, что значительно снижает количество ошибок времени выполнения и повышает надежность кода.
- Совместимость с JavaScript: возможность использования существующих JavaScript-библиотек и постепенного перехода к типизированному коду.

- Развитые инструменты разработки: автодополнение, рефакторинг и навигация по коду в современных IDE значительно превосходят возможности для обычного JavaScript.
- Официальная поддержка VS Code API: все типы и интерфейсы VS Code Extension API предоставляются в TypeScript, что упрощает разработку и снижает вероятность ошибок.
- Объектно-ориентированное программирование: поддержка классов, интерфейсов и модулей обеспечивает лучшую структуризацию кода сложных проектов.

1.1.3 Dart и Flutter Framework

Dart и Flutter представляют целевую экосистему для генерируемых тестов:

- Dart: современный объектно-ориентированный язык программирования, разработанный Google. Обеспечивает высокую производительность, поддержку асинхронного программирования и развитую систему типов.
- Flutter: кроссплатформенный фреймворк для разработки мобильных, веб и desktop приложений. Использует декларативный подход к построению пользовательского интерфейса и предоставляет богатый набор виджетов.
- Архитектурные паттерны: Flutter активно использует паттерны управления состоянием, такие как BLoC (Business Logic Component) и его упрощенная версия Cubit, которые являются основными объектами для генерации тестов в разработанном плагине.

1.1.4 Библиотека flutter_test

Библиотека flutter_test является основным инструментом для модульного тестирования во Flutter:

- Встроенная поддержка: входит в стандартный SDK Flutter, не требует дополнительной установки.
- Comprehensive API: предоставляет функции test(), group(), setUp(), tearDown() для структурирования тестов.
- Матчеры и assertions: богатый набор функций expect() с различными матчерами для проверки результатов тестирования.

- Поддержка асинхронного тестирования: встроенные механизмы для тестирования Future и Stream объектов.

1.1.5 Библиотека bloc_test

Специализированная библиотека bloc_test предназначена для тестирования BLoC и Cubit компонентов:

- Декларативный синтаксис: функция blocTest() предоставляет структурированный подход к описанию тестовых сценариев с блоками build, act, expect, verify.
- Тестирование последовательностей состояний: автоматическая проверка порядка эмиссии состояний в ответ на события или вызовы методов.
- Интеграция с mock-библиотеками: seamless работа с mockito для создания mock-объектов зависимостей.
- Поддержка сложных сценариев: возможность тестирования условных переходов, обработки ошибок и асинхронных операций.
- Отладочные возможности: детальный вывод информации о фактических и ожидаемых состояниях при сбое тестов.

Выбранный технологический стек обеспечивает создание надежного, поддерживаемого и легко расширяемого инструмента, полностью интегрированного в экосистему Flutter-разработки.

1.2 Тестирование в разработке на Flutter

Flutter, как современный фреймворк для создания кроссплатформенных приложений, предоставляет развитые инструменты для тестирования. Основными видами тестов являются модульные тесты, виджет-тесты и интеграционные тесты. Модульные тесты проверяют логику отдельных функций, методов или классов и выполняются быстрее всего. Для управления состоянием в Flutter-приложениях часто используется паттерн BLoC, в частности его упрощенная версия - Cubit. Тестирование Cubit включает проверку корректности переходов между состояниями в ответ на вызовы его методов. Пакет значительно упрощает написание таких тестов.

1.3 Проблемы ручного тестирования и необходимость автоматизации

Несмотря на наличие инструментов, ручное написание модульных тестов, особенно для Cubit, требует значительных временных затрат. Разработчику необходимо:

- Создать тестовый файл с корректной структурой и импортами.
- Для каждого значимого сценария описать последовательность действий (вызовов методов) и ожидаемых состояний.
- Учесть различные комбинации входных данных и начальных состояний.
- Поддерживать тесты в актуальном состоянии при изменении логики компонента.

Этот процесс не только трудоемок, но и подвержен человеческому фактору: возможен пропуск важных тестовых случаев или допущение ошибок в самих тестах. Автоматизация генерации базовых тестовых сценариев может существенно снизить эту нагрузку.

1.4 Постановка задачи разработки плагина

Задача состоит в создании расширения (плагина) для VS Code, которое позволит разработчикам генерировать заготовки модульных тестов для Dart-кода. Плагин должен:

1. Активироваться из контекстного меню редактора кода.
2. Анализировать выбранный класс Cubit/BLoC, DTO или метод.
3. Для Cubit/BLoC:
 - Идентифицировать его состояния (классы State).
 - Находить публичные методы и ивенты со связанными с ними приватными методами, изменяющие состояние (вызывающие emit).
 - Генерировать тестовые случаи с использованием bloc_test, покрывающие различные последовательности вызовов методов и переходов состояний.
4. Для DTO:

- Генерировать тестовые случаи, включая объявление Json и Entity, покрывающие такие случаи как: непришедшее поле, поле неправильно типа и лишние поля.
5. Для методов:
 - Генерировать шаблон теста, включая инициализацию класса (если метод не статический) и вызов метода с заглушками для аргументов и ожидаемого результата.
 6. Корректно формировать пути для тестовых файлов и необходимые импорты.
 7. Предоставлять сгенерированный код в виде, легко расширяемом и дорабатываемом пользователем.

2 Проектирование плагина

2.1 Архитектура плагина

Плагин разрабатывается как расширение для Visual Studio Code с использованием языка TypeScript. VS Code предоставляет API для взаимодействия с редактором, файловой системой и пользовательским интерфейсом.

Основные компоненты архитектуры плагина:

- Компонент инициализации (`extension.ts`): регистрирует команды плагина в VS Code и обрабатывает активацию расширения.
- Обработчик команд: активируется при вызове пользователем функции генерации тестов. Определяет контекст (выбранный файл, позиция курсора) для анализа.
- Лексический анализатор (`dartLexer.ts`): современный токенизатор Dart-кода, поддерживающий полную грамматику языка, включая сложные операторы, типы данных, строки с escape-последовательностями и импорты.
- Парсер Dart-кода: отвечает за анализ содержимого файла `.dart` для извлечения информации о классах, методах, состояниях Cubit/BLoC, импортах и структурных элементах.
- Генератор тестов для Cubit/BLoC (`cubitGenerator.ts`): на основе информации от парсера строит тестовые сценарии для Cubit/BLoC с поддержкой автоматов состояний и визуализации.
- Генератор тестов для DTO (`dtoGenerator.ts`): на основе информации от парсера строит тестовые сценарии парсинга JSON с автоматическим поиском связанных Entity.
- Генератор тестов для методов (`methodGenerator.ts`): создает шаблоны тестов для отдельных методов.
- Генератор веб-интерфейса (`webviewGenerator.ts`): создает интерактивные HTML-панели с визуализацией автоматов состояний, тестовых путей и детальной аналитикой в формате Graphviz.

- Модуль утилит (`utils.ts`): централизованные функции преобразования строк, константы состояний, регулярные выражения и общие вспомогательные функции.
- Модуль управления файлами: создает или обновляет тестовые файлы в правильной директории с автоматическим форматированием кода.

Система автоматической генерации тестов реализована в виде модульной архитектуры, состоящей из нескольких взаимосвязанных компонентов. Каждый компонент отвечает за определенный аспект анализа кода и генерации тестов, обеспечивая высокую степень разделения ответственности и возможность независимого тестирования модулей.

2.2 Основные интерфейсы и структуры данных

Архитектура системы базируется на наборе ключевых интерфейсов, определяющих структуру данных для представления различных аспектов анализируемого кода.

2.2.1 StateInfo — Информация о состояниях

```

1  interface StateInfo {
2      name: string;           // Название состояния
3      properties?: string[];  // Свойства состояния
4      isInitial?: boolean;    // Является ли начальным состоянием
5      isError?: boolean;      // Является ли состоянием ошибки
6      isLoading?: boolean;    // Является ли состоянием загрузки
7      isSuccess?: boolean;    // Является ли успешным состоянием
8  }
```

Интерфейс `StateInfo` представляет метаинформацию о состояниях Bloc/Cubit компонентов. Он включает классификацию состояний по типам (начальное, ошибка, загрузка, успех), что позволяет генератору тестов создавать более осмысленные тестовые сценарии.

2.2.2 EnhancedStateInfo — Расширенная информация о состояниях

```

1  interface EnhancedStateInfo extends StateInfo {
```

```

2     isReachable?: boolean;           // Достижимость состояния
3     reachabilityPaths?: string[];    // Пути достижения состояния
4 }

```

Расширенная версия интерфейса состояний, добавляющая информацию о достижимости состояний в рамках автомата состояний и возможных путях их достижения.

2.2.3 DependencyInfo — Информация о зависимостях

```

1 interface DependencyInfo {
2     name: string;                   // Имя переменной зависимости
3     type: string;                   // Тип зависимости
4 }

```

Интерфейс описывает зависимости Cubit/BLoC компонентов, такие как репозитории и сервисы, которые внедряются через конструктор и требуют создания mock-объектов в тестах.

2.2.4 EventInfo — Информация о событиях

```

1 interface EventInfo {
2     name: string;                   // Название события
3     handler: string;                 // Название метода-обработчика
4 }

```

Интерфейс описывает события в BLoC-архитектуре, устанавливая связь между событиями и их обработчиками.

2.2.5 ParsedMethod — Разобранный метод

```

1 interface ParsedMethod {
2     name: string;                   // Название метода/события
3     type: 'event' | 'method';       // Тип: событие (для BLoC) или метод
                                     // (для Cubit)
4     allowedFromStates: string[];    // Состояния, из которых можно вызвать
5     emitStates: string[];           // Состояния, которые эмитируются
6     guardConditions: string[];       // Guard условия (запрещенные состояния)
7 }

```

Базовая структура для представления методов и событий с основной информацией о переходах состояний.

2.2.6 EnhancedMethodInfo — Расширенная информация о методах

```
1 interface EnhancedMethodInfo {
2     name: string;           // Название метода/события
3     type: 'event' | 'method'; // Тип: событие (для Bloc) или метод
                               // (для Cubit)
4     event?: string;         // Само событие (для блоков)
5     handler?: string;       // Обработчик (для блоков)
6     parameters: string[];   // Параметры
7     transitions: StateTransition[]; // Детальные переходы между
                               // состояниями
8     reachableStates: string[]; // Все достижимые состояния из этого
                               // метода
9     finalStates: string[];   // Конечные состояния метода
10    transientStates: string[]; // Сквозные состояния метода
11    repositoryCalls: string[]; // Вызовы репозитория
12    storageCalls: string[];   // Вызовы хранилища
13    privateMethodCalls: string[]; // Вызовы частных методов
14    guardConditions: GuardCondition[]; // Guard условия
15    allowedStates: string[];   // Разрешенные состояния для вызова
16    isArrowFunction?: boolean; // Является ли стрелочной функцией
17 }
```

Детальная структура метода/события, включающая полную информацию о переходах состояний, вызовах зависимостей и ограничениях.

2.2.7 StateTransition — Переход между состояниями

```
1 interface StateTransition {
2     fromState: string;       // Начальное состояние
3     toState: string;         // Конечное состояние
4     transientStates: string[]; // Сквозные состояния между началом и
                               // концом
5     repositoryCalls: string[]; // Вызовы методов репозитория
6     storageCalls: string[];   // Вызовы методов хранилища
7     privateMethodCalls: string[]; // Вызовы частных методов
```

```

8     isReachable: boolean;           // Достижимость перехода
9     conditionGuards: string[];      // Условия/ограничения перехода
10  }

```

Детальное описание перехода между состояниями, включая информацию о промежуточных состояниях и вызовах внешних зависимостей.

2.2.8 GuardCondition — Guard условие

```

1  interface GuardCondition {
2      blockedState: string;           // Заблокированное состояние
3      action: string;                 // Действие (обычно "return")
4  }

```

Описывает ограничения на выполнение методов в определенных состояниях (например, ранние возвраты).

2.2.9 DartMethodContext — Контекст метода Dart

```

1  interface DartMethodContext {
2      blockType: 'try' | 'catch' | 'if' | 'else' | 'switch' | 'case' |
          'default' | 'root';
3      startIndex: number;
4      endIndex: number;
5      nestingLevel: number;
6      parentContext?: DartMethodContext;
7  }

```

Представляет контекст выполнения внутри методов Dart для анализа условных блоков и обработки исключений.

2.2.10 EmitInfo — Информация об emit вызове

```

1  interface EmitInfo {
2      state: string;                   // Нормализованное название состояния
3      rawStatement: string;           // Оригинальный emit statement
4      context: DartMethodContext | null;
5      isTransient: boolean;
6      isFinal: boolean;
7      reason: string;

```



```
8 }
```

Детальная информация о вызовах `emit()`, включая контекст и классификацию по типу (транзитное или финальное состояние).

2.2.11 FSM — Автомат конечных состояний

```
1 interface FSM {
2     states: string[];           // Все состояния в автомате
3     transitions: FSMTransition[]; // Все возможные переходы
4     initialState: string;       // Начальное состояние
5 }
6
7 interface FSMTransition {
8     from: string;               // Исходное состояние
9     to: string;                 // Целевое состояние
10    method: string;              // Метод/событие, вызывающее переход
11    condition?: string;          // Условие перехода
12 }
```

Формальная модель конечного автомата, построенного на основе анализа Bloc/Cubit компонента.

2.2.12 TestPath — Тестовый путь

```
1 interface TestPath {
2     states: string[];           // Последовательность состояний в пути
3     methods: string[];          // Последовательность методов/событий
4     conditions: (string | undefined)[]; // Условия для каждого перехода
5 }
```

Представляет тестовый сценарий как последовательность состояний и методов с условиями переходов.

2.2.13 ExecutionBranch — Ветка выполнения

```
1 interface ExecutionBranch {
2     branchId: string;           // Уникальный идентификатор ветки
3     states: string[];           // Последовательность состояний
4     repositoryCalls: string[];  // Вызовы методов репозитория
5 }
```

```

5     storageCalls: string[];           // Вызовы методов хранилища
6     privateMethodCalls: string[]; // Вызовы приватных методов
7     isSuccessPath: boolean;          // Путь успеха
8     isErrorPath: boolean;            // Путь с ошибкой
9     isFinalState: boolean;           // Конечное состояние
10 }

```

Описывает отдельную ветку выполнения в методе, включая информацию о вызовах внешних зависимостей, что необходимо для генерации правильных мок-настроек в тестах.

2.2.14 Token и TokenType — Лексический анализ

```

1  interface Token {
2      type: TokenType;
3      value: string;
4      position: number;
5      line: number;
6      column: number;
7  }
8
9  enum TokenType {
10     CLASS, IMPORT, FUNCTION, ASYNC, AWAIT, IF, ELSE, TRY, CATCH,
11     SWITCH, CASE, DEFAULT, RETURN, EMIT, IDENTIFIER, TYPE,
12     STRING, NUMBER, BOOLEAN, CHAR, FLOAT, DOUBLE,
13     ASSIGN, EQUALS, NOT_EQUALS, LT_EQUALS, GT_EQUALS,
14     LPAREN, RPAREN, LBRACE, RBRACE, LBRACKET, RBRACKET,
15     WHITESPACE, NEWLINE, COMMENT, EOF, UNKNOWN
16 }

```

Базовые структуры лексического анализатора для токенизации Dart-кода с поддержкой полной грамматики языка.

2.2.15 DataItem — Информация о полях DTO

```

1  interface DataItem {
2      dartName: string;           // Имя переменной в Dart-нотации
3      jsonKey: string;           // Имя переменной в JSON-нотации
4      type: string;              // Тип переменной
5  }

```

Специализированный интерфейс для генерации тестов DTO, описывающий маппинг между JSON-ключами и полями Dart-класса.

2.2.16 ASTNode — Узлы абстрактного синтаксического дерева

```
1  abstract class ASTNode {
2      public type: string;
3      constructor(type: string) { this.type = type; }
4  }
5
6  class ClassNode extends ASTNode {
7      constructor(
8          public name: string,
9          public extendsClass?: string,
10         public body: ASTNode[] = []
11     ) { super('class'); }
12 }
13
14 class MethodNode extends ASTNode {
15     constructor(
16         public name: string,
17         public returnType: string,
18         public isAsync: boolean = false,
19         public parameters: string[] = [],
20         public body: ASTNode[] = []
21     ) { super('method'); }
22 }
```

Базовые классы для представления элементов кода в виде абстрактного синтаксического дерева, используемые в перспективном AST-парсере.

2.3 Основные функциональные компоненты

2.3.1 Анализатор кода

Компонент анализа кода включает функции для статического анализа Dart-кода и извлечения структурной информации:

- `parseInlineStates()` — парсинг состояний, определенных в том же файле, что и Bloc/Cubit.

- `parseStates()` — анализ внешних файлов состояний.
- `parseCubitOrBloc()` — извлечение информации о методах, событиях и зависимостях.
- `parseMethodTransitions()` — анализ переходов состояний внутри методов.
- `parseRepositoryCalls()` и `parseStorageCalls()` — выявление вызовов внешних зависимостей.

2.3.2 Построитель автоматов

Модуль построения конечных автоматов преобразует результаты анализа кода в формальную модель:

- `buildFSM()` — основная функция построения автомата состояний из проанализированных компонентов.
- `analyzeStates()` — классификация состояний на транзитные и финальные.
- `analyzeEmitContext()` — анализ контекста вызовов `emit` для определения условий переходов.

2.3.3 Генератор тестовых путей

Компонент для генерации тестовых путей на основе теории графов:

- `generateTestPaths()` — основная функция генерации путей с ограничением по количеству.
- `buildBFSSpanningTree()` — построение остовного дерева обходом в ширину.
- `generateInTreePaths()` — генерация путей в остовном дереве.
- `generateOutTreePaths()` — генерация путей для рёбер вне остовного дерева.
- `removeDuplicatePaths()` — удаление дублирующихся путей.

2.3.4 Генератор тестового кода

Модуль трансляции тестовых путей в исполняемый код тестов:

- `generateBlocTests()` — основная функция генерации тестов для Bloc/Cubit.
- `generateMinimalRequiredTests()` — генерация обязательных тестов для всех методов.
- `generateSingleMethodTest()` — генерация теста для отдельного метода.
- `generateMockSetup()` — создание настроек mock-объектов.
- `generateStateInstance()` — генерация экземпляров состояний.

2.3.5 Интеграция с VS Code

Компонент интеграции обеспечивает взаимодействие с API редактора:

- `generateTest()` — главная экспортируемая функция плагина.
- `generateFeatureTestFile()` — создание структуры тестов на уровне feature.
- `updateMainTestFile()` — обновление главного файла тестов проекта.
- `applyAllCodeActions()` — применение автоматического форматирования.
- `runTestsForFile()` — запуск сгенерированных тестов.

2.3.6 Лексический анализатор Dart

Система лексического анализа представляет собой современный токенизатор, реализующий полную поддержку грамматики Dart:

- Поддержка операторов: Полная поддержка всех операторов сравнения (`<=`, `>=`, `!=`, `==`), арифметических операторов и операторов присваивания.
- Типы данных: Распознавание всех примитивных типов Dart (`int`, `double`, `String`, `bool`, `char`, `float`) и их вариантов.
- Обработка строк: Корректная обработка строковых литералов с escape-последовательностями (`\n`, `\t`, `\"`) и различными типами кавычек.
- Числовые литералы: Поддержка положительных, отрицательных чисел, чисел с плавающей точкой, суффиксов типов (`f`, `d`) и различных форматов записи.

- Ключевые слова: Распознавание всех ключевых слов Dart, включая модификаторы доступа, управляющие конструкции и объявления типов.
- Импорты: Специальная обработка `import` выражений для корректного построения зависимостей между модулями.
- Комментарии: Поддержка как однострочных (`//`), так и многострочных (`/* */`) комментариев с возможностью их игнорирования или сохранения.

Лексер построен на основе конечного автомата, обеспечивающего высокую производительность и точность анализа даже для сложных конструкций Dart-кода.

2.4 Порядок выполнения

Общий порядок выполнения системы включает следующие этапы:

1. Инициализация: Пользователь активирует команду генерации тестов в VS Code.
2. Анализ кода: Система анализирует выбранный файл и связанные компоненты.
3. Построение модели: На основе анализа строится автомат состояний (для Bloc/Cubit) или извлекается структурная информация (для DTO/методов).
4. Генерация путей: Для автоматов генерируются тестовые пути с использованием алгоритма остовного дерева.
5. Создание тестов: Пути транслируются в исполняемый код тестов с автоматической настройкой mock-объектов.
6. Интеграция: Созданные тесты интегрируются в структуру проекта с обновлением главных тестовых файлов.

2.5 Взаимодействие с пользователем

Плагин интегрируется в VS Code через контекстное меню, появляющееся при вызове контекстного меню (`Ctrl+Shift+.` / `Cmd+.`) на названии класса Cubit/BLoC, DTO или метода в редакторе кода. Команда «Flutter: Generate test» инициирует процесс анализа и генерации.

Результатом является создание структуры тестовых файлов в соответствующих директориях проекта. При генерации тестов для Cubit/BLoC плагин дополнительно создает интерактивную веб-панель (webview), которая предоставляет:

- Визуализацию автомата состояний: Интерактивный граф в формате Graphviz, показывающий все состояния компонента и возможные переходы между ними.
- Отображение тестовых путей: Граф тестовых множеств с цветовой дифференциацией путей и использованием различной толщины линий для уникальной идентификации каждого пути.
- Детальный анализ методов: Сворачивающиеся секции с информацией о каждом методе/событии, включая список достижимых состояний, вызовы зависимостей и ветки выполнения.
- Информацию о ветках выполнения: Структурированное отображение всех найденных путей выполнения с указанием типа пути (успех/ошибка) и связанных вызовов.
- Копирование в буфер обмена: Возможность копирования DOT-кода графов для использования в внешних инструментах визуализации.
- Интеграция с Graphviz Online: Прямая ссылка на онлайн-сервис для редактирования и расширенной визуализации графов.

Веб-интерфейс использует современные технологии HTML5, CSS3 и JavaScript для обеспечения удобного взаимодействия и высокой информативности отображаемых данных.

2.6 Парсинг исходного кода

Парсинг Dart-кода осуществляется с использованием комбинации лексического анализа и целевых регулярных выражений для идентификации ключевых конструкций:

- Лексический анализ: Входной код разбивается на токены с использованием специализированного лексера, поддерживающего полную грамматику Dart, включая сложные операторы, типы данных и строковые литералы.
- Определение структуры классов: Извлечение информации о классе Cubit/BLoC, его базовом классе состояния и иерархии наследования.

- Анализ состояний: Поиск всех классов состояний, наследующих базовое состояние, с определением их свойств, конструкторов и семантических характеристик.
- Парсинг методов и событий: Для Cubit анализируются публичные методы, для BLoC - события и их обработчики, с извлечением полной сигнатуры и параметров.
- Анализ emit-вызовов: Детальный разбор тел методов для обнаружения всех вызовов `emit()`, определения контекста (условные блоки, `try-catch`) и классификации состояний.
- Трассировка зависимостей: Поиск и анализ вызовов репозиториев, сервисов и приватных методов для автоматической генерации mock-настроек.
- Анализ импортов: С использованием лексера извлекаются все импорты файла, включая относительные пути и псевдонимы, для корректного формирования зависимостей в тестах.
- Определение Guard-условий: Поиск ограничений на вызовы методов (например, ранние возвраты при определенных состояниях).

Данный подход обеспечивает высокую точность анализа и устойчивость к различным стилям написания кода, что критически важно для корректности генерируемых тестов.

3 Реализация ключевых функций

3.1 Генерация тестов для Cubit/BLoC

Процесс генерации тестов для Cubit/BLoC является наиболее сложным компонентом плагина и включает несколько этапов анализа и генерации.

3.1.1 Полный алгоритм работы с Cubit/BLoC

Этап 1: Парсинг и извлечение структурных элементов

Плагин выполняет комплексный анализ исходного кода для извлечения следующих элементов:

- Идентификация классов состояний: Парсер ищет все классы, наследующие базовое состояние (State). Для каждого состояния определяются его свойства, конструкторы и семантические характеристики (является ли состояние начальным, ошибочным, состоянием загрузки или успеха).
- Анализ зависимостей: Извлекаются все зависимости Cubit/BLoC (репозитории, сервисы), которые внедряются через конструктор. Это необходимо для последующего создания mock-объектов.
- Парсинг методов/ивентов: Для Cubit анализируются публичные методы, для BLoC - события (Event) и соответствующие им методы-обработчики. Определяются параметры каждого метода/ивента и их типы.
- Анализ тел методов/обработчиков: Детальный разбор кода методов для обнаружения вызовов `emit()`, условных конструкций, блоков `try-catch`, вызовов зависимостей.

Этап 2: Построение модели конечного автомата

На основе извлеченной информации строится формальная модель поведения:

- Определение множества состояний S : Все классы состояний становятся узлами автомата.
- Определение алфавита входных сигналов: Методы/ивенты формируют алфавит переходов.
- Построение функции переходов: Для каждого метода/ивента анализируются возможные переходы между состояниями через вызовы `emit()`.

- Выявление guard-условий: Определяются ограничения на переходы (например, метод может быть вызван только в определенных состояниях).
- Классификация состояний: Состояния маркируются как транзитные (проходные) или финальные (завершающие) на основе анализа контекста их использования.

Этап 3: Построение тестовых множеств

Плагин применяет комбинацию алгоритмов для генерации полного набора тестовых путей:

- BFS-обход для построения остовного дерева: Строится минимальное покрывающее дерево автомата, обеспечивающее достижимость всех состояний.
- Генерация базовых путей: Для каждого состояния строится путь от начального состояния.
- Выявление дополнительных рёбер: Находятся переходы, не включенные в остовное дерево.
- Генерация комбинированных путей: Строятся сложные тестовые сценарии, покрывающие последовательности переходов.
- Удаление дубликатов: Исключаются избыточные тестовые пути.

Этап 4: Анализ ветвей выполнения

Для каждого метода/ивента выполняется детальный анализ:

- Парсинг условных конструкций: Выявляются ветки if-else, switch-case.
- Анализ блоков try-catch: Определяются пути успешного выполнения и обработки ошибок.
- Трассировка вызовов зависимостей: Отслеживаются вызовы методов репозитория и сервисов.
- Построение графа потока управления: Для сложных методов строится детальная модель выполнения.

Этап 5: Трансляция в конечные тесты

Каждый тестовый путь транслируется в код теста `blocTest`:

- Генерация блока `build`: Создается экземпляр `Cubit/BLoC` с mock-зависимостями.

- Настройка mock-объектов: Для каждого пути настраиваются соответствующие ответы mock-зависимостей (успешные результаты или исключения).
- Формирование блока act: Генерируется последовательность вызовов методов/отправки ивентов с типовыми параметрами.
- Построение блока expect: Определяется ожидаемая последовательность состояний.
- Добавление блока verify: Проверяется корректность вызовов mock-зависимостей.

3.2 Генерация тестов для Data Transfer Object

Генерация тестов для объектов передачи данных (Data Transfer Objects) реализована в `dtoGenerator.ts` и включает следующие этапы:

3.2.1 Парсинг структуры Data Transfer Object

1. Анализ метода `fromJson`: Плагин парсит конструктор `fromJson` для извлечения маппинга между JSON-ключами и полями Dart-класса. Определяются типы полей через анализ `as Type` конструкций.
2. Определение связанного Entity: Анализируется метод `toEntity()` для определения целевого класса-сущности, в который преобразуется DTO.
3. Построение модели данных: Создается внутреннее представление с информацией о каждом поле: имя в Dart-нотации, ключ в JSON, тип данных.

3.2.2 Генерация тестовых сценариев

Для каждого DTO генерируется комплект тестов, покрывающих различные сценарии парсинга JSON:

- Тест с корректными данными: Проверяется успешный парсинг JSON с правильной структурой и типами полей.
- Тест с лишними полями: Убеждается, что дополнительные поля в JSON не нарушают парсинг.

- Тесты на отсутствие обязательных полей: Для каждого поля генерируется тест, проверяющий обработку ситуации, когда поле отсутствует в JSON.
- Тесты на несоответствие типов: Для каждого поля проверяется поведение при получении значения неправильного типа.

3.2.3 Структура генерируемых тестов

Каждый тестовый файл содержит:

- Константу с тестовыми данными `_<dto_name>Data`
- Ожидаемый объект `Entity _expected`
- Группу тестов `group('Сценарий парсинга <DtoName>')`
- Набор тестовых случаев с использованием `expect` и проверкой исключений `throwsA(isA<TypeError>())`

3.3 Генерация тестов для методов/ивентов

Для обычных методов классов (не Cubit/BLoC) или top-level функций, `methodGenerator.ts` генерирует упрощенный шаблон теста:

1. Определение сигнатуры метода/ивента: Извлекается имя метода/ивента, его параметры и их типы, а также тип возвращаемого значения.
2. Создание тестового случая: Генерируется функция `test` из пакета `test`.
3. Инициализация: Если метод принадлежит классу, предусматривается место для инициализации экземпляра класса.
4. Вызов метода/ивента: Генерируется вызов метода/ивента с заглушками для аргументов.
5. Проверка результата: Добавляется `expect` с заглушкой для ожидаемого результата, которую пользователь должен будет заполнить.

Этот подход предоставляет разработчику каркас, который необходимо дозаполнить конкретными тестовыми данными и ожиданиями.

3.4 Решение проблем и доработки

В ходе разработки были решены следующие ключевые задачи:

3.4.1 Создание стабильной архитектуры

- Модульная архитектура: Разделение функциональности на независимые модули (`dartLexer.ts`, `webviewGenerator.ts`, `utils.ts`) для улучшения поддерживаемости и тестируемости кода.
- Централизация утилит: Создание модуля `utils.ts` с общими функциями преобразования строк, константами состояний и регулярными выражениями для предотвращения дублирования кода.

3.4.2 Лексического анализа

- Расширение грамматики: Реализована поддержка всех операторов и большинства типов данных.
- Обработка числовых литералов: Реализован корректный парсинг положительный, отрицательных чисел и чисел с плавающей точкой, включая суффиксы типов.
- Строковый анализ: Реализована обработка строковых литералов с `escape`-последовательностями и различными типами кавычек.
- Составные операторы: реализована поддержка сложных операторов (`=>`, `!`) и их корректная токенизация.

3.4.3 Технические решения

- Корректное определение путей для импорта: Реализована логика для определения относительных путей к тестируемым файлам из тестовых файлов с поддержкой различных структур проектов.
- Обработка сложных типов параметров: Реализован парсинг для определения типов аргументов методов/событий Cubit/BLoC для генерации корректных заглушек и mock-объектов.
- Анализ `emit`-контекста: Детальный анализ контекста вызовов `emit()` для определения транзитных и финальных состояний.

3.4.4 Визуализация и пользовательский интерфейс

- Построение графов состояний: Реализованы алгоритмы для визуализации автомата состояний и тестовых путей в формате Graphviz с поддержкой цветовой дифференциации.
- Интерактивный webview: Создание современного веб-интерфейса с возможностью копирования графов, сворачивающимися секциями и интеграцией с внешними сервисами.
- Уникальная идентификация путей: Использование комбинации цветов и толщины линий для визуального различения тестовых путей в легенде.

3.4.5 Качество кода

- Современная документация: Использован современный формат TypeScript JSDoc для всех функций и интерфейсов.
- Типизация: Полная типизация всех функций и интерфейсов для предотвращения ошибок времени выполнения.

4 Инструкция пользователя

Данный раздел содержит пошаговое руководство по установке и использованию разработанного плагина для автоматической генерации тестов в среде VS Code.

4.1 Установка плагина

1. Скачайте файл плагина с расширением `.vsix` или выполните команду `npm run compile` из директории плагина и запустите проект нажатием F5 для режима разработки.
2. В VS Code откройте панель расширений через меню Extensions или используйте сочетание клавиш `Ctrl+Shift+X`.
3. Нажмите на значок «...» (три точки) в верхней части панели расширений.
4. Выберите пункт «Install from VSIX...».
5. Укажите путь к скачанному файлу плагина и подтвердите установку.

4.2 Подготовка проекта

Для корректной работы плагина убедитесь, что ваш Flutter-проект имеет следующую структуру директорий:

```
lib/  
— features/  
    — [название_фичи]/  
        * bloc/ (или cubit/)  
            · [имя]_bloc.dart  
            · [имя]_event.dart (только для BLoC)  
            · [имя]_state.dart  
— domain/  
    — repositories/  
    — services/
```

Эта структура обеспечивает автоматическое определение компонентов и правильное размещение генерируемых тестов.

4.3 Использование плагина

1. Откройте файл с компонентом, для которого необходимо сгенерировать тесты (BLoC, Cubit, DTO или файл с методом). Например, откройте файл `auth_login_bloc.dart`.
2. Установите курсор на название класса в начале файла (именно на объявление класса).
3. Нажмите сочетание клавиш `Ctrl+Shift+.` или `Cmd+.` для открытия контекстного меню.
4. В строке поиска введите «Flutter: Generate Test» и выберите соответствующую команду.
5. Плагин автоматически выполнит анализ компонента и создаст структуру тестов.

4.4 Результат генерации

После выполнения команды плагин создаст следующую структуру тестов:

```
test/  
— features/  
    — [название_фичи]/  
        * bloc/ (или cubit/)  
            · [имя]_bloc(или cubit)_test.dart — тест  
              кубита/блока  
        * dto  
            · [имя]_dto_test.dart — тест dto  
        * [имя]_test.dart — общий тест фичи  
— unit_test/  
    — method1/  
    — method2/  
— test.dart — общий тест всех фичей
```

4.5 Возможности генерируемых тестов

Плагин создает тесты со следующими характеристиками:

- Полные unit-тесты с автоматически сгенерированными mock-объектами (возможны ручные доработки для специфических случаев).
- Покрытие всех методов/событий, которые изменяют состояние компонента.
- Проверка начального состояния и корректности инициализации.
- Комбинированные тесты, состоящие из последовательных вызовов нескольких методов или событий.
- Визуализация автомата состояний и тестовых множеств для компонентов с состоянием.
- Автоматическое форматирование сгенерированного кода согласно стандартам Dart.

5 Результаты работы

Разработанный плагин для VS Code успешно автоматизирует генерацию модульных тестов для различных компонентов Flutter-приложений. Результаты работы продемонстрированы на реальных примерах сгенерированных тестов.

5.1 Результаты генерации тестов для Cubit

Для класса `NotificationToggleCubit` плагин сгенерировал комплексный тестовый файл, включающий:

- Проверку начального состояния: Тест корректности установки `NotificationInitialState` при создании экземпляра.
- Базовые тесты переходов: Для каждого метода (`enableNotifications`, `disableNotifications`, `enablePushOnly`, `enableEmailOnly`, `resetNotifications`) сгенерированы индивидуальные тесты с проверкой эмиссии соответствующих состояний.
- Комбинированные тесты: Сгенерированы пять множественных тестов, проверяющих последовательности методов, такие как `enableNotifications -> disableNotifications` и `enableNotifications -> enablePushOnly`.
- Автоматические mock-объекты: Созданы mock-классы для зависимостей `ISecureStorageService` и `INotificationService` с корректной настройкой и очисткой в блоках `setUp` и `tearDown`.
- Функцию генерации тестовых данных: Добавлена функция `generateTestData()` для создания типовых данных.

Результат выполнения, в виде всплывающей информационной панели, представлен в Приложении Б.

5.2 Результаты генерации тестов для BLoC

Для класса `AuthLoginBloc` плагин продемонстрировал способность работы с паттерном BLoC, генерируя тесты для событий:

- Тесты для событий авторизации: Сгенерированы тесты для `AuthLoginUserEvent` с двумя сценариями — успешной авториза-

ции (переход `AuthInitialLoginState` -> `AuthLoadingLoginState` -> `AuthSuccessLoginState`) и ошибки авторизации (переход в `AuthErrorLoginState`).

- Тесты валидации: Созданы тесты для событий `AuthValidateEmailEvent` и `AuthValidatePasswordEvent`, проверяющие переход в `AuthValidationLoginState`.
- Тесты управления состоянием: Включены тесты для `AuthResetStateEvent` и `AuthLogoutEvent` с различными сценариями выполнения.
- Mock-настройки репозиториев: Автоматическая настройка mock-объектов для `IAuthRepository` и `ISecureStorageService` с корректными `when` и `verify` блоками.
- Комбинированные сценарии: Множественные тесты, объединяющие различные события в логические последовательности.

Результат выполнения, в виде всплывающей информационной панели, представлен в Приложении А.

5.3 Результаты генерации тестов для Data Transfer Object

Для класса `AuthLoginDto` плагин сгенерировал специализированные тесты парсинга JSON:

- Базовый тест корректного парсинга: Проверка успешного преобразования JSON с полями `access_token` и `refresh_token` в объект `AuthLoginEntity`.
- Тест устойчивости к лишним полям: Проверка, что дополнительные поля в JSON не нарушают процесс парсинга.
- Тесты на отсутствие обязательных полей: Для каждого поля (`access_token`, `refresh_token`) сгенерированы тесты, проверяющие выброс `TypeError` при отсутствии поля.
- Тесты на несоответствие типов: Для каждого поля созданы тесты, проверяющие поведение при получении значения неправильного типа (например, числа вместо строки).

- Структурированные тестовые данные: Константа `_authlogindtoData` с типовыми значениями и объект `_expected` для сравнения результатов.

5.4 Качественные характеристики генерируемых тестов

Анализ сгенерированных тестов показывает следующие достоинства:

- Полнота покрытия: Тесты покрывают все публичные методы/события и основные сценарии их использования.
- Корректная структура: Все тесты следуют паттерну AAA (Arrange-Act-Assert) и используют соответствующие фреймворки (`bloc_test` для Cubit/BLoC, стандартный `test` для DTO).
- Правильные импорты: Автоматически добавляются все необходимые импорты, включая тестируемые классы, `mock`-библиотеки и тестовые фреймворки.
- Описательные имена тестов: Каждый тест имеет информативное название, указывающее на тестируемый сценарий.
- Готовность к использованию: Сгенерированные тесты могут быть запущены сразу, либо после небольших корректировок пользователем, в случае наличия сложных структур в методах.

5.5 Практическая ценность результатов

Использование плагина демонстрирует следующие преимущества:

- Экономия времени разработчика: Автоматическая генерация базовой структуры тестов сокращает время написания тестов, в среднем, на 3-4 часа, учитывая отсутствие необходимости ручного написания и изучения структуры.
- Стандартизация подходов: Все тесты следуют единому стилю и используют лучшие практики тестирования Flutter.
- Снижение барьера входа: Начинающие разработчики получают готовые примеры корректного написания тестов.

- Улучшение покрытия кода: Автоматическая генерация тестов позволяет достигать большего покрытия и предоставляет возможность быстрой актуализации тестов после внесения изменений в тестируемые структуры.

ЗАКЛЮЧЕНИЕ

В рамках данной работы был разработан и описан плагин для Visual Studio Code, предназначенный для автоматизации генерации модульных тестов для Flutter-приложений. Основное внимание было уделено генерации тестов для компонентов управления состоянием Cubit/BLoC, DTO и для отдельных методов.

Плагин успешно реализует полный цикл анализа и генерации тестов, включающий:

- Современный лексический анализ: Реализован полнофункциональный токенизатор Dart-кода, поддерживающий сложные языковые конструкции, операторы и типы данных.
- Интеллектуальный анализ архитектуры: Автоматическое построение моделей конечных автоматов для Cubit/BLoC компонентов с определением состояний, переходов и ограничений.
- Комплексная генерация тестов: Создание полноценных тестовых наборов с автоматической настройкой mock-объектов, покрытием всех методов/событий и генерацией комбинированных сценариев.
- Интерактивную визуализацию: Веб-интерфейс с графами автоматов состояний, тестовых путей и детальной аналитикой для лучшего понимания логики компонентов.
- Модульную архитектуру: Разделение функциональности на независимые, хорошо документированные модули с централизованными утилитами.

Ключевые достижения проекта:

- Создание устойчивого к различным стилям кода анализатора с поддержкой полной грамматики Dart.
- Реализация алгоритмов построения тестовых множеств на основе теории графов и остовных деревьев.
- Разработка интуитивного пользовательского интерфейса с современными веб-технологиями.
- Обеспечение высокого качества генерируемого кода с автоматическим форматированием и соответствием лучшим практикам.

- Создание расширяемой архитектуры, позволяющей легко добавлять поддержку новых типов компонентов и паттернов.

Использование данного плагина способствует снижению трудоемкости процесса написания модульных тестов, повышает скорость разработки и помогает поддерживать качество кодовой базы и ее отказоустойчивость на должном уровне. Несмотря на существующие ограничения, связанные, в основном, с простотой используемого парсера и требованиями к структуре проекта, плагин представляет собой полезный инструмент для Flutter-разработчиков.

Возможные направления дальнейшего развития проекта:

- AST-парсер: Переход на использование полноценного Abstract Syntax Tree парсера для еще более точного анализа кода.
- Оптимизация тестовых путей: Внедрение алгоритмов машинного обучения для выбора наиболее значимых тестовых сценариев.
- Расширение покрытия: Поддержка дополнительных паттернов управления состоянием (Riverpod, Redux, GetX).
- CI/CD интеграция: Автоматическое обновление тестов при изменениях в исходном коде через GitHub Actions или подобные системы.
- Метрики качества: Добавление анализа покрытия кода и предложений по улучшению тестовых сценариев.
- Командная работа: Интеграция с системами code review для автоматического предложения тестов в Pull Request.

Предложенное решение представляет собой значительный шаг в автоматизации рутинных задач разработки и может служить основой для создания более комплексных инструментов статического анализа и автоматического тестирования в экосистеме Flutter.

ПРИЛОЖЕНИЕ А

В данном приложении представлены изображения генеративного информационного файла, при вызове генерации тестов для `auth_login_bloc`.

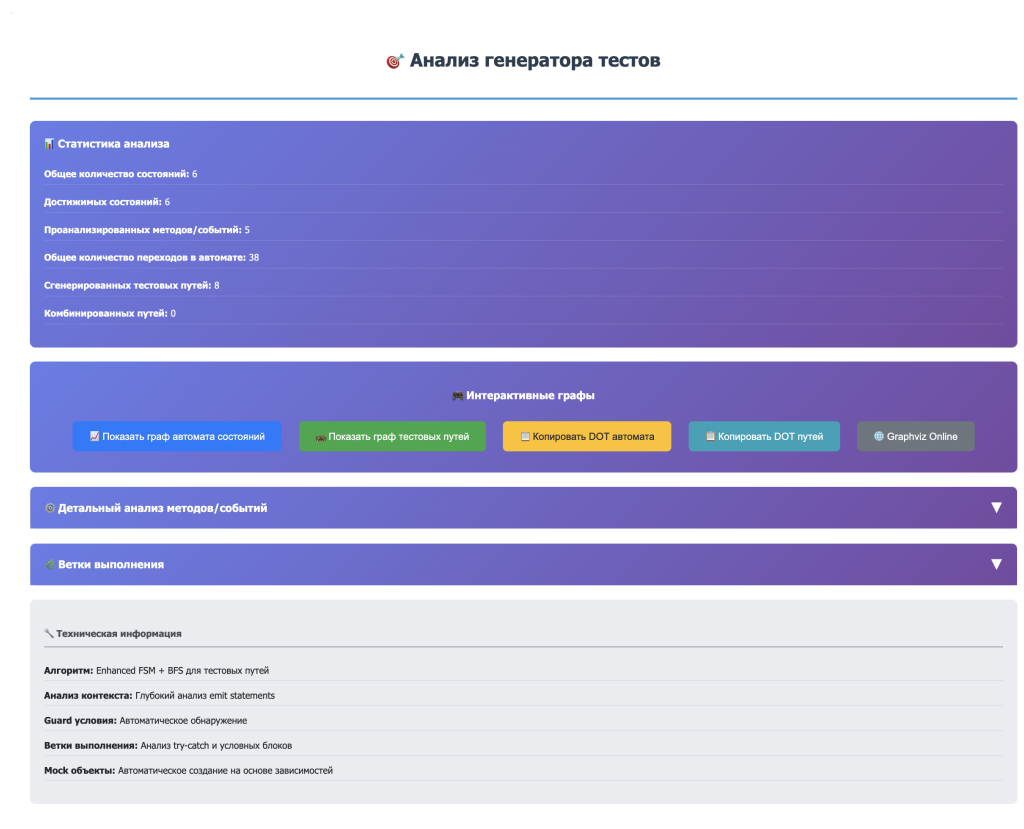


Рисунок 1 — Общая информация о блоке.

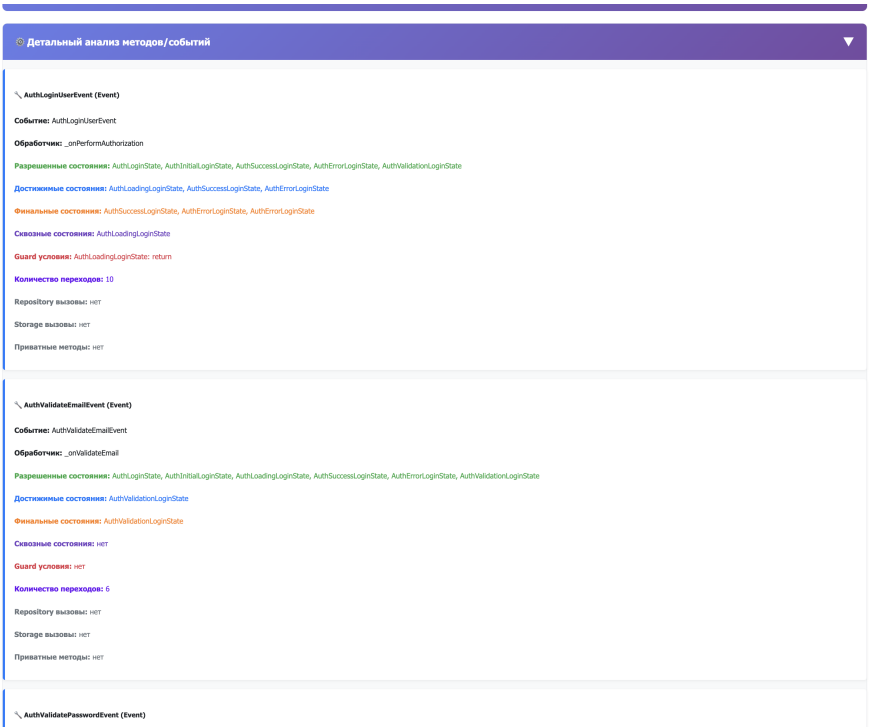


Рисунок 2 — Информация об ивентах блока.

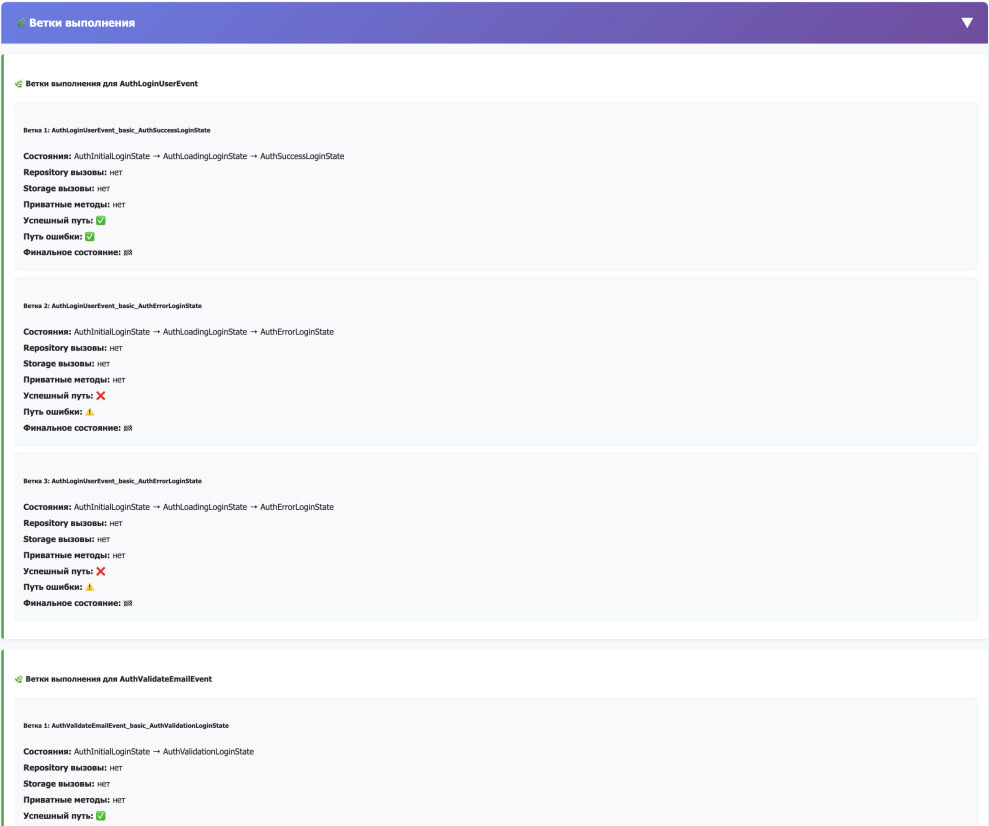


Рисунок 3 — Информация о тестах блока.

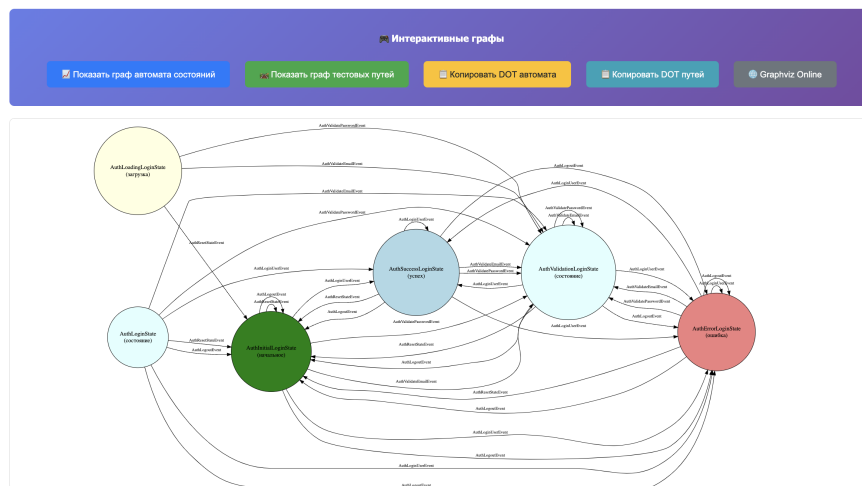


Рисунок 4 — Атомат состояний блока.

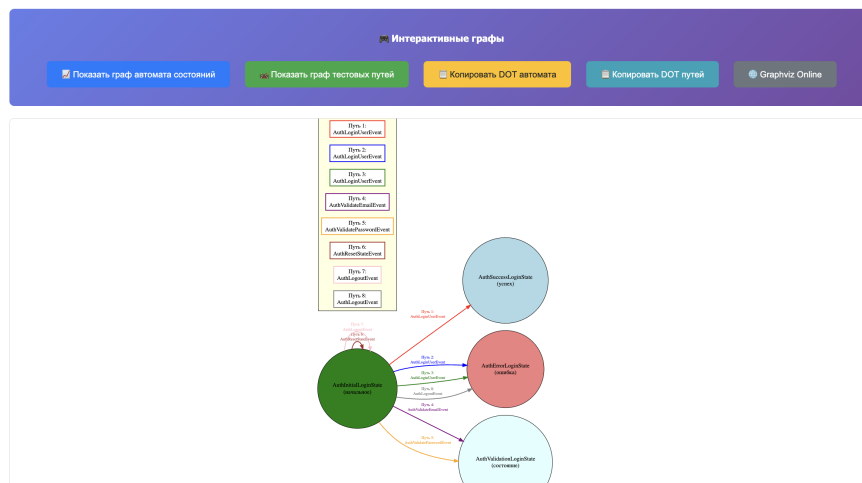


Рисунок 5 — Тестовые множества блока.

ПРИЛОЖЕНИЕ Б

В данном приложении представлены изображения генеративного информационного файла, при вызове генерации тестов для `notification_toggle_cubit`.

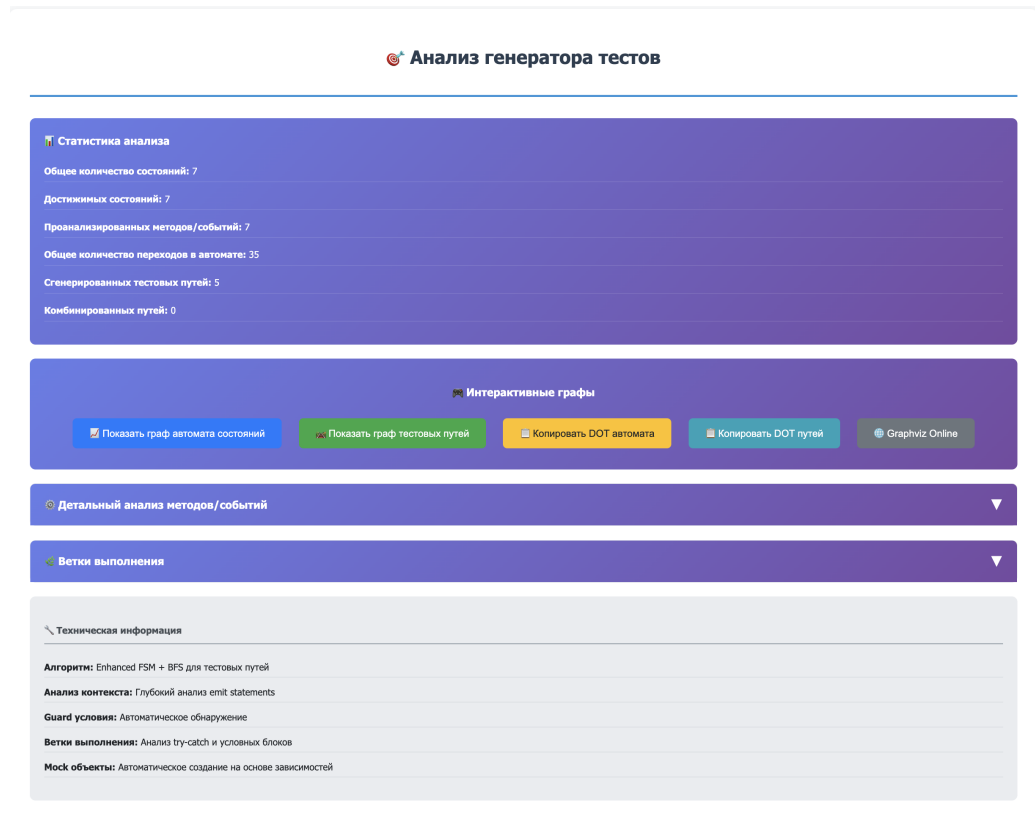


Рисунок 6 — Общая информация о кубите.

Детальный анализ методов/событий
\. enableNotifications (Method) Разрешенные состояния: NotificationToggleState, NotificationInitialState, NotificationEnabledState, NotificationDisabledState, NotificationPushEnabledState, NotificationEmailEnabledState, NotificationLoadingState Достижимые состояния: NotificationEnabledState Финальные состояния: NotificationEnabledState Скользящие состояния: нет Guard условия: нет Количество переходов: 7 Repository вызовы: нет Storage вызовы: нет Приватные методы: нет Тип: Стрелочная функция
\. disableNotifications (Method) Разрешенные состояния: NotificationToggleState, NotificationInitialState, NotificationEnabledState, NotificationDisabledState, NotificationPushEnabledState, NotificationEmailEnabledState, NotificationLoadingState Достижимые состояния: NotificationDisabledState Финальные состояния: NotificationDisabledState Скользящие состояния: нет Guard условия: нет Количество переходов: 7 Repository вызовы: нет Storage вызовы: нет Приватные методы: нет Тип: Стрелочная функция
\. enablePushOnly (Method) Разрешенные состояния: NotificationToggleState, NotificationInitialState, NotificationEnabledState, NotificationDisabledState, NotificationPushEnabledState, NotificationEmailEnabledState, NotificationLoadingState Достижимые состояния: NotificationPushEnabledState Финальные состояния: NotificationPushEnabledState Скользящие состояния: нет Guard условия: нет Количество переходов: 7 Repository вызовы: нет Storage вызовы: нет Приватные методы: нет Тип: Стрелочная функция

Рисунок 7 — Информация о методах кубита.

Ветки выполнения
Ветки выполнения для enableNotifications Ветка 1: enableNotifications_basic_NotificationEnabledState Состояния: NotificationInitialState → NotificationEnabledState Repository вызовы: нет Storage вызовы: нет Приватные методы: нет Успешный путь: ☒ Путь ошибки: ☒ Финальное состояние: ☒
Ветки выполнения для disableNotifications Ветка 1: disableNotifications_basic_NotificationDisabledState Состояния: NotificationInitialState → NotificationDisabledState Repository вызовы: нет Storage вызовы: нет Приватные методы: нет Успешный путь: ☒ Путь ошибки: ☒ Финальное состояние: ☒
Ветки выполнения для enablePushOnly Ветка 1: enablePushOnly_basic_NotificationPushEnabledState Состояния: NotificationInitialState → NotificationPushEnabledState Repository вызовы: нет Storage вызовы: нет Приватные методы: нет Успешный путь: ☒ Путь ошибки: ☒ Финальное состояние: ☒
Ветки выполнения для enableEmailOnly

Рисунок 8 — Информация о тестах кубита.

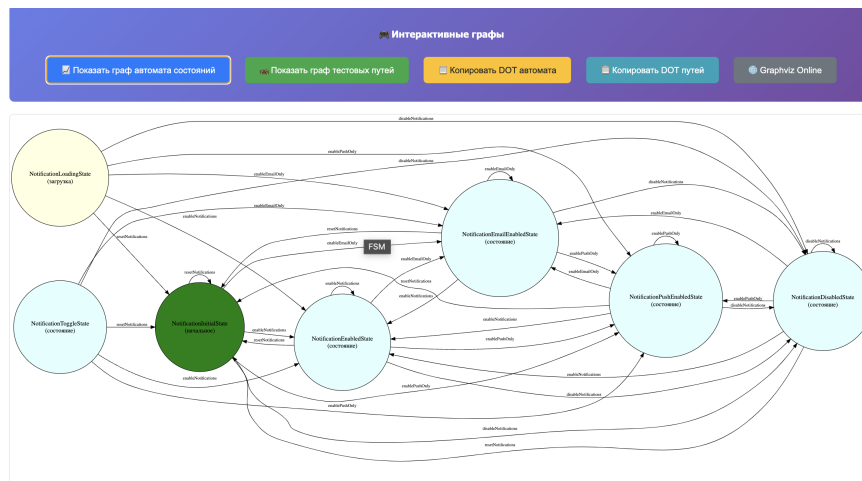


Рисунок 9 — Атомат состояний кубита.

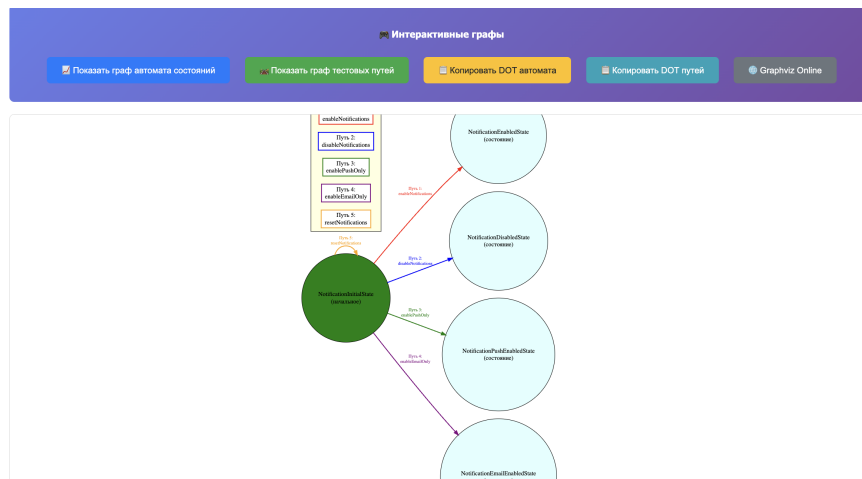


Рисунок 10 — Тестовые множества кубита.